Predicting LLM Pretraining Loss Curves Across Learning-Rate Schedules

TDLT Task 2 Final Project

Group members: TODO names, student IDs, graduate/undergraduate status

Peking University

June 19, 2026

Reading Guide

These slides are written as a standalone report.

- Sections 1–2 define the prediction problem and the course protocol.
- Sections 3–4 describe reproduced baselines and our method-development attempts.
- Sections 5–6 give quantitative results, qualitative curves, and failure analysis.
- The appendix records formulas, reproducibility details, and caveats.

Project Requirement

TDLT Task 2 asks us to study loss-curve prediction for LLM pretraining under learning-rate schedules.

- Use the provided course loss curves.
- Fit a model on the **cosine** learning-rate schedule.
- Evaluate prediction performance on the **WSD** schedule.
- Reproduce existing approaches and develop/evaluate modified fitting strategies.
- Submit code in a reproducible repository and include the repository link in the slides.

Repository link placeholder: `TODO: add GitHub URL after upload`.

Why Predict Loss Curves?

Loss-curve prediction tries to answer a practical planning question before paying for a full training run.

- If we know a loss curve under one schedule, can we predict another schedule?
- If schedule choice affects extrapolation, scaling-law fits are not only about model/data size.
- Accurate prediction can reduce expensive trial-and-error in pretraining schedule design.

The key challenge is that a learning-rate schedule affects the entire optimization history, not only the current step.

Learning-Rate Schedule As Input

For total training length T, define a nonnegative learning-rate vector

$$\boldsymbol{\eta} = (\eta_1, \eta_2, \ldots, \eta_T)^\top \in \mathbb{R}^T_{\geq 0}.$$

The pretraining loss at step t is treated as a functional of the schedule prefix:

$$L_t = L_\Theta(\eta_{\leq t}), \qquad \eta_{\leq t} = (\eta_1, \eta_2, \ldots, \eta_t).$$

We fit Θ from observed loss curves and use it to predict unseen schedules.

Course Protocol

The strict main protocol is:

$$\text{fit on cosine LRS} \quad \longrightarrow \quad \text{predict WSD LRS.}$$

- Fit target: EMA-smoothed loss with span 201.
- Evaluation region: post-warmup, implemented as `step >= 1000`.
- Evaluation grid: every 20 recorded steps plus the final point.
- Auxiliary schedule: 8-1-1, used only for additional evidence.

Main Evaluation Metric

The headline number is WSD full-region RMSE on the EMA target:

$$\mathrm{RMSE} = \left(\frac{1}{|I|} \sum_{t \in I} \left(\widehat{L}_t - L_t^{\mathrm{EMA}} \right)^2 \right)^{1/2}.$$

We also report:

- MAE and MAPE for average absolute error.
- R^2 for explained variance.
- FinalE and WorstE for endpoint and worst-case behavior.
- Tail20/decay/final1000 regional errors as diagnostic views.

Central Difficulty

The data regime is extremely small.

- Only one cosine curve is available for the main fit.
- WSD is the main test curve, but it is tempting to use it during method development.
- 8-1-1 is useful, but it is not part of the official main protocol.

This makes overparameterized formulas fragile: a lower cosine fitting loss can easily hurt schedule transfer.

Course Data

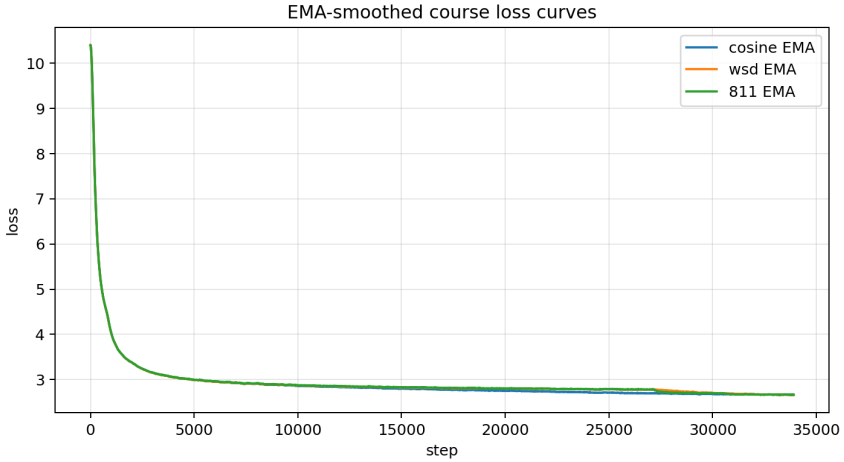
The file `data/loss_curves/gpt_loss+lrs.pkl` contains three pandas DataFrames.

Schedule	Role	Points	Approx. max step
cosine	main fit	33907	33906
WSD	main test	33907	33906
8-1-1	auxiliary unseen	33908	33907

Required columns are `step`, `Metrics/loss`, and `lr`.

Learning-Rate Schedules

Observed Loss Curves



Indexing Choices

Canonical strict configuration:

Parameter	Value
EMA span	201
min step	1000
fit stride	800
tail fit stride	250
eval stride	20
source stride for MPL/FSL-MPL+	100
MTL restarts	20

All generated CSVs store the full predictions used to recompute metrics.

Data Manifest And Integrity Checks

The cleaned repository writes a data manifest for each run.

- Confirms schedule name, row count, step range, peak LR, final LR, final raw/EMA loss.
- Uses the explicit course `lr` column instead of reconstructing schedules from filenames.
- Keeps the original pickle unchanged under `data/loss_curves/`.

Sanity checks verify finite positive losses, finite learning rates, and recomputable metrics.

Train/Test Leakage Policy

We distinguish three levels of evidence.

- **Clean strict reproduction**: fit and model selection use cosine only.
- **Auxiliary evidence**: evaluate on 8-1-1 without using it for main selection.
- **WSD-adaptive development**: use WSD RMSE to search hyperparameters; valuable but not an unbiased WSD test.

The final tuned MTL result belongs to the third category and is labeled that way throughout.

Baseline 1: Momentum Law (MTL)

The Tissue et al. learning-rate annealing law is implemented as

$$\widehat{L}(t) = L_0 + A\, T_t^{-\alpha} - C\, S_2(t), \qquad T_t = \epsilon + \sum_{i \le t} \eta_i.$$

The annealing memory is

$$m_i = \lambda m_{i-1} + (\eta_{i-1} - \eta_i), \qquad S_2(t) = \sum_{i \le t} m_i.$$

We grid-search λ and fit (L_0, A, C, α) by log-Huber loss.

MTL Strengths And Weaknesses

Strengths

- Few parameters and stable fitting.
- Strong inductive bias for schedule memory.
- Best clean baseline on WSD EMA full RMSE.

Weaknesses

- A single exponential memory scale can miss multi-timescale schedule effects.
- Grid choice for λ strongly affects transfer.

Baseline 2: Multi-Power Law (MPL)

MPL models schedule response with a nonlinear accumulated LR-drop term:

$$\widehat{L}(t) = L_0 + AT_t^{-\alpha} - B\sum_{i\le t}\Delta_i^{+}\left[1-\left(1+C\,\eta_i^{-\gamma}(T_t - T_i + \eta_i)\right)^{-\beta}\right].$$

Here $\Delta_i^{+} = \max(\eta_{i-1} - \eta_i, 0)$ extracts positive LR decay sources.

MPL Implementation Adaptation

The original MPL repository could not be used directly on the course CSV form.

- Its loader reconstructs learning-rate schedules from hard-coded paper filenames.
- The cleaned implementation uses the course `lr` column directly.
- Original-compatible adapter checks showed the formula/evaluator agrees numerically with the upstream evaluator when given the same parameters.

This separates a data-adapter problem from the mathematical model.

Baseline 3: Faithful FSL

Functional Scaling Laws use intrinsic time:

$$T(k) = \sum_{i \le k} \eta_i.$$

The practical LLM ansatz from Appendix B.2 is

$$L_\Theta(k) = L_0 + \frac{c_1}{T(k)^s} - \mathrm{LRD}(k),$$

where LRD is a schedule-response convolution over LR drops.

FSL Practical Response

In fixed model-size course data, the implemented response is

$$\mathrm{LRD}(k) = c_2 \sum_{i \le k} (\eta_{i-1} - \eta_i)^+ \left(c_3 + \frac{1}{T(i)^s} \right) \left(1 - \frac{1}{(1 + c_4(T(k) - T(i)))^\gamma} \right).$$

We do not invent missing kernel-regression quantities such as spectra, source coefficients, or noise variance.

FSL Reproduction Policy

Faithful FSL is freshly fit from the audited reproduction formula.

- The code contains the FSL fitter and runs it by default.
- The final table uses a fixed-seed 10-restart fit to control local optimizer drift.
- The fitted model uses cosine only; WSD loss is used only for evaluation.

This keeps the result generated from code while preserving the formula implementation.

Unified Baseline Outputs

Every baseline writes the same result schema:

- `predictions.csv`: method, schedule, role, step, lr, raw/EMA loss, prediction, errors.
- `metrics.csv`: protocol, method, schedule, target, region, R^2, MAE, RMSE, FinalE, WorstE.
- `fitted_params.json`: parameters, fit target, source stride, and selection-signal notes.

Development Goal

We explored methods that keep the same input/output protocol but change the schedule-response bias.

- Can MPL-style power response improve noisy raw-loss behavior?
- Can FSL-style intrinsic-time kernels improve transfer?
- Can multiple memory scales beat single-λ MTL?
- Can a tuned MTL optimizer/hyperparameter setting recover missed performance?

FSL-MPL+ Small

FSL-MPL+ small replaces MPL's response with a simpler intrinsic-time power response:

$$R_{\text{small}}(t,i) = 1 - \Big(1 + \kappa \eta_i^{-\gamma}(T_t - T_i)\Big)^{-q}.$$

Prediction form:

$$\widehat{L}(t) = L_0 + A T_t^{-\alpha} - B \sum_i \Delta_i^+ R_{\text{small}}(t,i).$$

The intent is to reduce purely empirical coupling while retaining LR-drop response.

FSL-MPL+ Source

The source variant adds a source-amplitude term:

$$\widehat{L}(t) = L_0 + AT_t^{-\alpha} - B\sum_i \Delta_i^+ \left(1 + c_s T_i^{-\alpha}\right) R_{\text{small}}(t, i).$$

Motivation:

- LR drops at different intrinsic times may have different effective strength.
- The source term borrows FSL's view that earlier/later sources have different decay behavior.

Kernelized MTL

KMTL replaces one exponential memory scale by a nonnegative mixture:

$$S_K(t) = \sum_{j=1}^{m} w_j S_{\lambda_j}(t), \qquad w_j \geq 0, \quad \sum_j w_j = 1.$$

Tested fixed kernels:

- $m = 2$: $\{0.99, 0.999\}$.
- $m = 3$: $\{0.99, 0.999, 0.9999\}$.

The hope was to approximate heavy-tail memory with a small mixture.

NCPL-Lite Residual

NCPL-lite is a small residual diagnostic, not a full neural ansatz.

$$\log L_{\mathrm{true}}(t) - \log L_{\mathrm{base}}(t) \approx r_\theta(\phi(t)).$$

Implementation:

- Fit FSL-MPL+ source first.
- Fit a ridge residual on cosine features only.
- Clip residual corrections to prevent runaway extrapolation.

Tuned MTL

The final tuned method keeps the MTL formula unchanged.

- Wider λ grid includes $\lambda = 0.9997$.
- Tail-weighted cosine fitting uses tail weight 3.0.
- Same four fitted parameters: (L_0, A, C, α).

Tuned parameters:

$$L_0 = 2.7284, \quad A = 774.10, \quad C = 4.93 \times 10^{-5}, \quad \alpha = 0.9343, \quad \lambda = 0.9997.$$

Tuned MTL Disclosure

Tuned MTL is numerically strongest, but its selection is not a clean WSD holdout.

- The autoresearch loop evaluated WSD RMSE repeatedly.
- Attempts were kept/discarded based on WSD improvement.
- Therefore WSD acted as an adaptive development signal.

We present it as an important development result: the original MTL family was under-optimized.

What We Did Not Ship

The final repository does not include the raw 296-attempt autoresearch loop.

- Raw journals are useful during discovery but noisy for a final project repository.
- The final repo ships tuned-MTL search-setting metadata, not tuned fitted weights.
- Baselines, variants, FSL, NCPL-lite, and tuned MTL are freshly fit with fixed seeds.

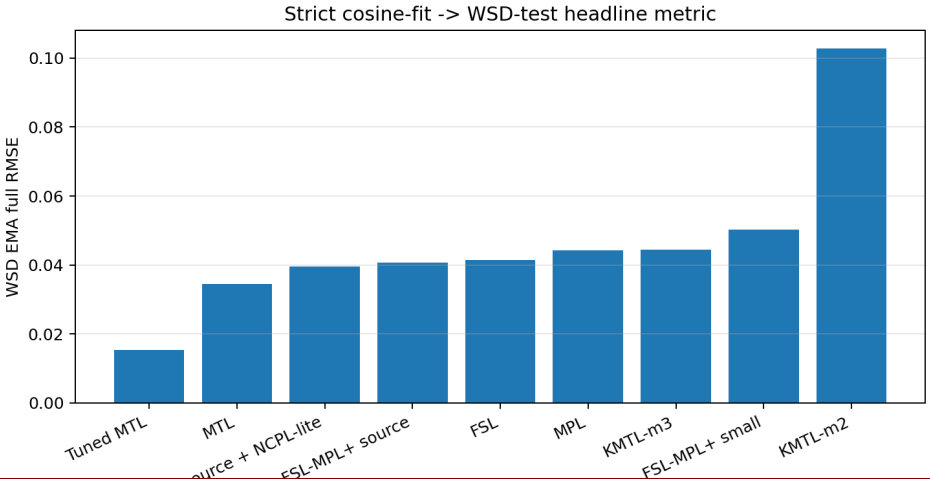
This keeps the repo presentable while preserving the result logic.

Headline Table: WSD EMA Full

Method	RMSE	MAE	R^2	FinalE
Tuned MTL	0.015461	0.010069	0.992727	0.016741
MTL	0.034569	0.028816	0.963640	0.004317
FSL-MPL+ source + NCPL-lite	0.038326	0.027214	0.955309	0.081484
FSL-MPL+ source	0.038635	0.026214	0.954584	0.027232
MPL	0.040219	0.031148	0.950783	0.029069
FSL	0.040358	0.035311	0.950443	0.053288
KMTL-m3	0.044395	0.035417	0.940033	0.064200
FSL-MPL+ small	0.050309	0.037628	0.922992	0.011666
KMTL-m2	0.102856	0.090321	0.678114	0.050873

Tuned MTL is best numerically; clean MTL is the best non-WSD-adaptive baseline.

Headline RMSE Bar Chart



Main Interpretation

Three main messages:

- MTL is a very strong clean baseline in the one-cosine-fit regime.
- More expressive variants often improve one metric but worsen another.
- Tuned MTL cuts WSD EMA full RMSE by about 55% relative to clean MTL, but this gain is WSD-adaptive.

The important scientific point is not only "which row wins", but why small-data schedule transfer punishes extra flexibility.

Tuned MTL Versus Clean MTL

Metric	Clean MTL	Tuned MTL	Change
WSD EMA full RMSE	0.034569	0.015461	-55.3%
WSD EMA full MAE	0.028816	0.010070	-65.1%
WSD EMA full MAPE	0.010213	0.003591	-64.8%
WSD FinalE	0.004317	0.016748	worse
WSD WorstE	0.063370	0.082274	worse

Tuned MTL improves the full-curve objective but spends some error at the final point.

Auxiliary 8-1-1 Evidence

8-1-1 was not used for the tuned-MTL keep/discard rule.

Method	8-1-1 EMA full RMSE	Interpretation
Tuned MTL	0.013594	strong auxiliary transfer
MTL	0.030190	clean baseline
FSL-MPL+ source + NCPL-lite	0.035313	lower MAE, worse final
MPL	0.041732	weaker transfer

This supports, but does not prove, that the tuned basin is meaningful beyond WSD.

Raw-Loss Metrics

Raw loss is noisier than EMA loss.

- Some FSL-MPL+ variants show competitive raw-loss behavior.
- EMA remains the main metric because it reduces observation noise and matches the report protocol.
- A method can look good on raw MAE but still fail tail/final diagnostics.

In our final story, raw metrics are supporting diagnostics, not the headline.

Final-Step Tradeoff

Tuned MTL improves the full WSD region but has larger final error than clean MTL.

- Clean MTL FinalE: 0.004317.
- Tuned MTL FinalE: 0.016700.
- NCPL-lite FinalE: 0.083302.

If the project objective were "last point only", the ranking would change.

Worst-Case Error Tradeoff

WorstE highlights local failures:

Method	WSD EMA WorstE
MTL	0.063370
Tuned MTL	0.082189
FSL	0.069316
NCPL-lite	0.286106
MPL	0.313013

This is why the slides report multiple metrics instead of one leaderboard.

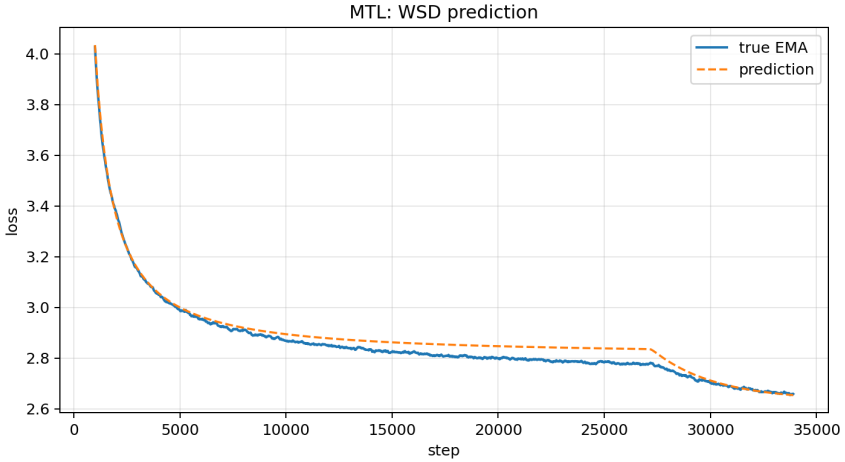
Region-Level Lesson

Region metrics reveal why "more expressive" is not automatically better.

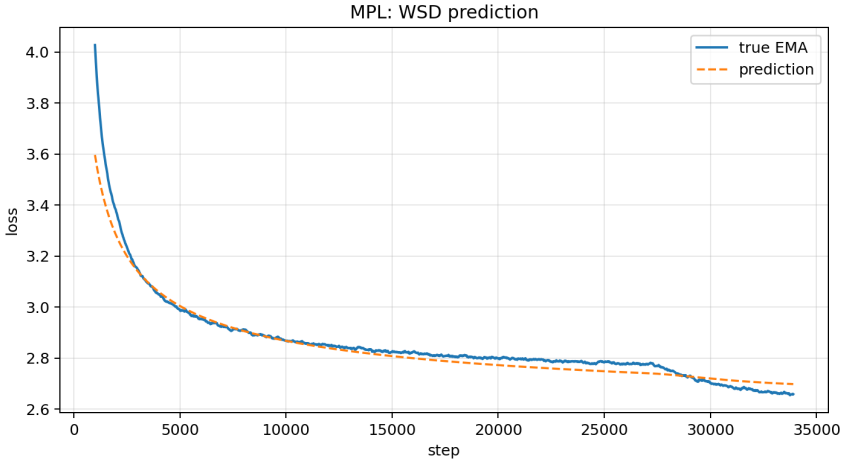
- Tail20 has low variance, so R^2 can be unstable or negative even when absolute error is moderate.
- Full-region RMSE rewards the long stable/early-decay region.
- Final1000 and FinalE expose endpoint calibration failures.

The tuned MTL result is best for the declared full-region objective, not all objectives.

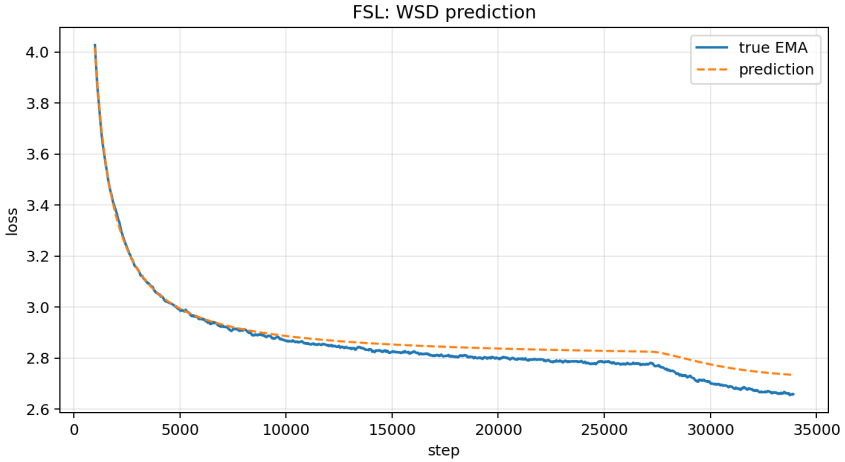
Clean MTL WSD Prediction



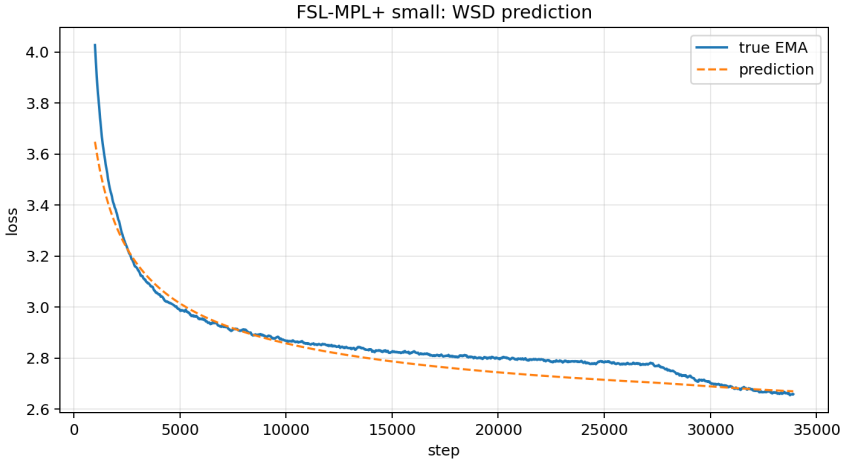
MPL WSD Prediction



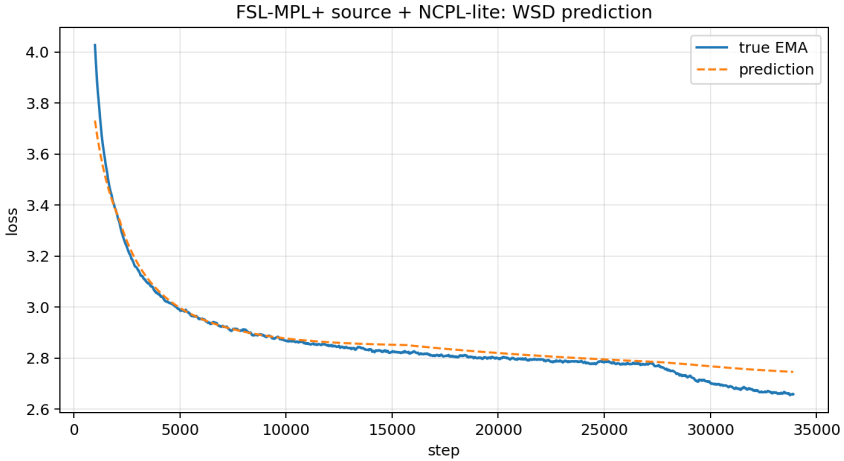
Faithful FSL WSD Prediction



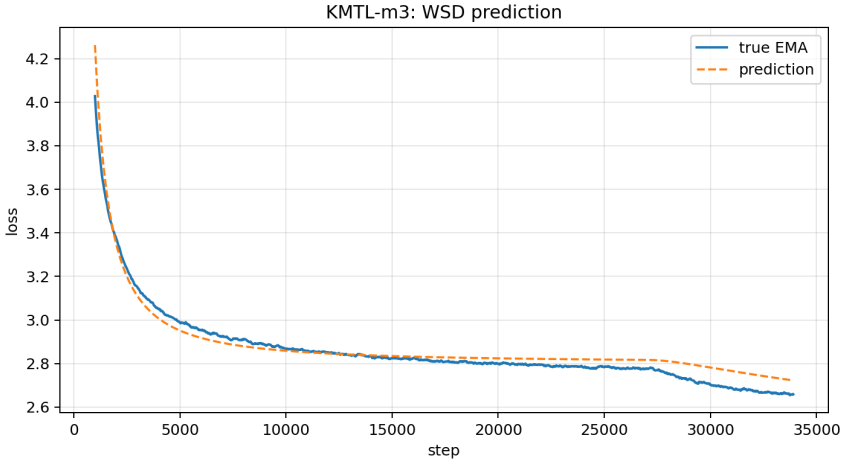
FSL-MPL+ Small WSD Prediction



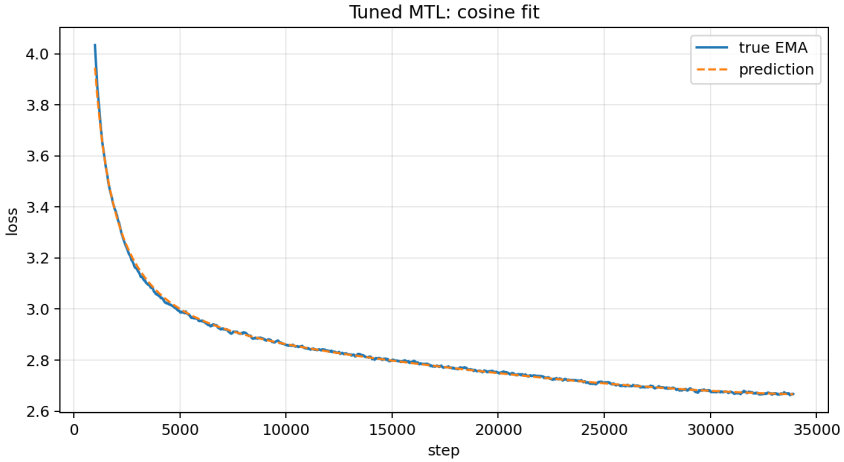
NCPL-Lite WSD Prediction



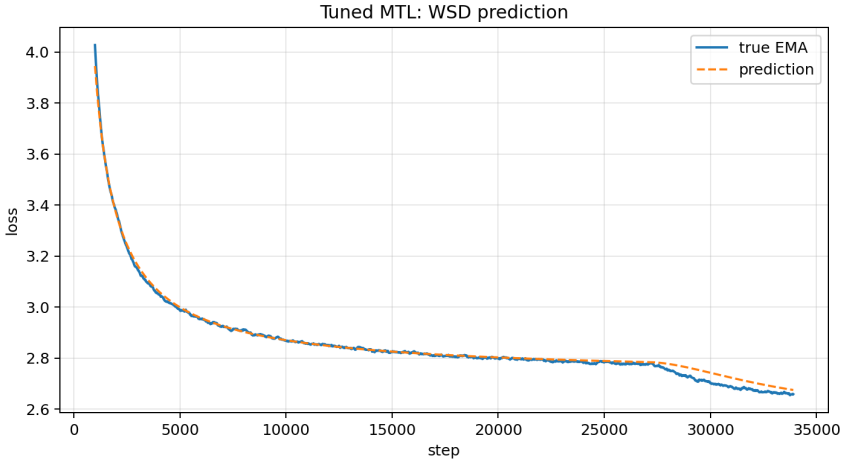
KMTL-m3 WSD Prediction



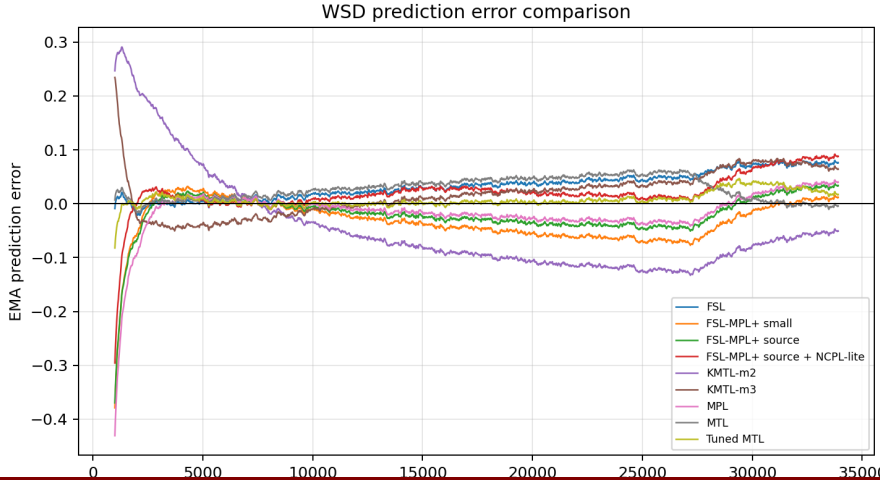
Tuned MTL Cosine Fit



Tuned MTL WSD Prediction



All-Method WSD Error Curves



Failure Pattern 1: More Restarts Alone

The autoresearch logs show that simply increasing MTL restarts does not explain the final gain.

- 50, 100, and 200 restarts with the default grid stay near the original MTL basin.
- The key was the wider λ grid plus a stronger tail-weighted fit.

Lesson: optimizer effort helps only if the search space contains the useful basin.

Failure Pattern 2: Overexpressive Schedule Response

MPL and FSL-MPL+ introduce extra nonlinear degrees of freedom.

- These can reduce some average errors.
- But they often produce larger endpoint or worst-case errors.
- With one fit curve, parameter identifiability is the bottleneck.

Failure Pattern 3: Kernelized MTL Instability

KMTL tries to approximate multi-timescale memory.

- $m = 2$ can collapse into poor local minima.
- $m = 3$ has better final error but worse full RMSE than MTL.
- Mixture weights are hard to identify from one cosine curve.

Failure Pattern 4: Residual Correction Overreach

NCPL-lite has the lowest non-tuned MAE but poor final/worst errors.

- Residual features learn cosine residual structure.
- The learned residual does not extrapolate cleanly to WSD tail/final behavior.
- Clipping prevents catastrophic blow-up but does not solve calibration.

Failure Pattern 5: Tail Weight Sensitivity

Tail weighting matters.

- Low tail weight can overfit the broad stable region.
- High tail weight can improve WSD transfer for MTL.
- Too much tail emphasis can hurt final or tail-specific objectives.

Tuned MTL uses tail weight 3.0 because the WSD-adaptive search found it effective for full RMSE.

What The Failures Teach

The negative results are useful.

- Schedule-transfer prediction is strongly constrained by data scarcity.
- The best formula is not necessarily the most expressive formula.
- Hyperparameter grids can dominate the apparent performance of a simple model.
- A transparent leakage policy is necessary when method search touches WSD.

Clean Repository Layout

- `data/loss_curves/`
- `src/tdlt_losscurves/`
- `src/tdlt_losscurves/models/`
- `scripts/run_baselines.py`
- `scripts/run_variants.py`
- `scripts/run_tuned_mtl.py`
- `scripts/reproduce_all.py`
- `configs/`
- `results/`
- `docs/`
- `slides/`
- `tests/`

Environment

The environment was created as:

Commands

```
conda create -y -n tdlt python=3.10
pip install numpy==1.26.4 scipy==1.11.4 matplotlib==3.8.4 tqdm==4.66.4
scikit-learn==1.4.2 pandas==2.2.2
pip install torch==2.2.2 -index-url https://download.pytorch.org/whl/cpu
```

Import check records torch as `2.2.2+cpu`.

Reproduction Commands

Full run

```
conda run -n tdlt python scripts/reproduce_all.py -out-dir results
```

Individual runs

```
conda run -n tdlt python scripts/run_baselines.py
conda run -n tdlt python scripts/run_variants.py
conda run -n tdlt python scripts/run_tuned_mtl.py
```

Code Correctness Checks

Fast checks cover:

- Constant LR gives zero MTL annealing memory.
- KMTL with one kernel matches MTL.
- FSL response is zero when there are no LR-drop sources.
- FSL response is monotone with intrinsic-time gap.
- RMSE is recomputable from `predictions.csv`.

The result folder also stores `results/env/import_check.txt` and `pip_freeze.txt`.

Git History

The final repository has a staged history:

- scaffold and provenance;
- reusable package and CLIs;
- tuned-selection metadata and fixed-seed fresh-fit configs;
- regenerated results;
- Beamer slides and rendered QA;
- final documentation polish.

This makes the cleanup process auditable rather than a single opaque copy.

Limitations

- Only three course curves are available.
- Main protocol has one fit schedule, so complex formulas are underidentified.
- Tuned MTL uses WSD-adaptive model selection.
- We do not claim a new theoretical law; the contribution is reproducible comparison and a transparent tuning finding.

Conclusion

- MTL is the best clean non-WSD-adaptive baseline under the strict protocol.
- MPL/FSL/FSL-MPL+/KMTL/NCPL-lite expose useful modeling directions but are not uniformly better on this tiny data regime.
- Tuned MTL achieves the strongest WSD full-region RMSE by finding a better λ basin and tail weight.
- The tuned result should be read as WSD-adaptive development, with 8-1-1 as auxiliary support.

Group Information And Division Of Labor

- Member 1: TODO name, student ID, graduate/undergraduate status.
- Member 2: TODO name, student ID, graduate/undergraduate status.
- Member 3: TODO name, student ID, graduate/undergraduate status.

Division of labor placeholder:

- Literature/formula audit: TODO.
- Code/reproduction: TODO.
- Method development/results/slides: TODO.

9 Appendix

Appendix: MTL Lambda Grids

Clean MTL grid:

$$\{0.95,\ 0.99,\ 0.995,\ 0.999,\ 0.9995\}.$$

Tuned MTL wider grid:

$$\{0.9, 0.95, 0.97, 0.99, 0.993, 0.995, 0.997, 0.999, 0.9993, 0.9995, 0.9997, 0.9999, 0.99995\}.$$

The selected tuned value is $\lambda = 0.9997$.

Appendix: Tuned MTL Parameters

Parameter	Value
L_0	2.7284210565
A	774.0970766973
C	4.9317543×10^{-5}
α	0.9342665162
λ	0.9997
objective	0.0001109515

Appendix: Original-Compatible Reproduction

Original-compatible MTL/MPL adapters were kept as provenance, not copied wholesale.

- MTL original-compatible and clean wrapper predictions agree up to numerical roundoff.
- MPL original evaluator agrees with the clean wrapper when using the same parameters.
- The main obstacle to direct original-code execution was hard-coded data loading, not formula mismatch.

Appendix: Why Pandas Is In The Environment

The course file is a pandas pickle:

`gpt_loss+lrs.pkl`.

The course readme explicitly says to read it with pandas. Therefore the final environment adds:

`pandas==2.2.2`.

This is the only dependency added beyond the requested scientific stack.

Appendix: References

- Tissue, Wang, and Wang. *Scaling Law with Learning Rate Annealing*. 2024.
- Luo et al. *A Multi-Power Law for Loss Curve Prediction Across Learning Rate Schedules*. 2024.
- Li, Chen, Huang, Wang, and Wu. *Functional Scaling Laws in Kernel Regression: Loss Dynamics and Learning Rate Schedules*. 2025.
- Zhang, Wen, and Ma. *Configuration-to-Performance Scaling Law with Neural Ansatz*. 2026.
- Hu et al. *MiniCPM: Unveiling the Potential of Small Language Models with Scalable Training Strategies*. 2024.
- Bi et al. *DeepSeek LLM: Scaling Open-Source Language Models with Longtermism*. 2024.